

INTER-PROCESS COMMUNICATION (IPC): A process is a running program. Sometimes two or more processes need to communicate or share data

Topic :

- Message Queues
- Shared Memory

Message Queues :

A message queue is an IPC mechanism in which processes exchange data as messages stored in a queue maintained by the operating system kernel. The sender puts a message into the queue and the receiver picks it up later.

- Messages stay in the queue until a process reads them, so sender and receiver need not run at the same time.
- Messages are normally read in FIFO order, but can carry a type/priority so a receiver can select specific messages.
- Can be used between related or unrelated processes, since the queue is identified by a key/name.

Example:

Process A sends the message "Hello B" to a message queue. Process B reads the message from the same queue.

Process A -> Message Queue -> Process B

Linux example: msgget(), msgsnd(), msgrcv(), msgctl()

View queues: ipcs -q

Shared Memory :

Shared memory is an IPC mechanism in which a block of memory is mapped into the address space of two or more processes, so they can read and write the same data directly.

- It is the fastest IPC method, because data is not copied through the kernel after the memory is set up.
- Processes must be synchronized (using semaphores or mutexes) to avoid reading and writing at the same time.
- The segment stays in memory until it is explicitly removed.

Example:

Process A writes "Good Morning" into a shared memory segment. Process B attaches to the same segment and reads the message.

Process A -> Shared Memory <- Process B

Linux example: shmget(), shmat(), shmdt(), shmctl()

View segments: ipcs -m